

Creative Revenue & Rights Dashboard

Project Documentation

Author: Jeevesh Singale
Brand: LUNIM
Version: 3.0
Date: May 2026
Status: Production-Ready

1. Project Overview

The **Creative Revenue & Rights Dashboard** is a full-stack Web3 platform built for entertainment companies. It enables administrators to manage creative projects (web series, films, music productions), assign percentage-based rights to contributors, and distribute revenue in real-time via Ethereum smart contracts.

Core Problem: In the entertainment industry, revenue distribution across multiple rights holders is opaque, slow, and dispute-prone.

Solution: A trustless, transparent smart-contract-backed platform where every stakeholder can monitor earnings through a secure, real-time dashboard.

2. Technology Stack

Layer	Technology
Frontend	Next.js 14 (App Router), React 18, TypeScript
Styling	Tailwind CSS 3, Framer Motion
Charts	Chart.js + react-chartjs-2
Backend	Node.js + Express.js
Database	Supabase (PostgreSQL)
Authentication	Supabase Auth (email/password)
Blockchain	Ethereum via Hardhat (local node, Chain ID 31337)
Web3 Library	Ethers.js v6
Wallet	MetaMask + WalletConnect v2
Smart Contracts	Solidity ^0.8.20
Notifications	react-hot-toast

3. Application Pages & Features

3.1 Landing Page (/)

A cinematic dark-themed hero page with animated gradient mesh backgrounds. Displays:

- "The Standard For Creative Rights" headline

- "Live Network Sync" status indicator
- Footer showing network status (Ethereum Mainnet Simulated)
- Login/signup CTAs; auto-redirects authenticated users to `/dashboard`

3.2 Login Page (`/login`)

Split-screen layout with branding on the left and sign-in form on the right:

- Email and password authentication via Supabase Auth
- Auto-fill button for demo admin credentials
- Animated gradient backgrounds with Framer Motion
- Auto-redirects to `/dashboard` on successful login

3.3 Dashboard (`/dashboard`)

The main workspace after login. Features a **sticky header** with:

- **Project Switcher:** Dropdown to select individual projects or "All Projects"
- **Live Sync Indicator:** Green pulse showing real-time data connection
- **Stat Pills:** Total Distributed, Rights Holders count, Transactions count, Project Status

The dashboard has **5 tabs** (admin sees all; non-admins see Overview and Revenue only):

Overview Tab

- **Charts Panel** — Revenue analytics via Chart.js
- **Recent Activity** — Latest platform events
- **Top Rights Holders** — Top 3 holders with name, role, percentage, and total received
- **Add Holder** button (admin only, for specific projects)

Revenue Tab

- **Revenue Snapshot** — Detailed transaction list with expandable split breakdowns
- Each transaction shows: project name, tx hash, total amount (ETH + USD), status badge
- Expanding a transaction reveals per-holder splits with animated slide-down (Framer Motion)
- Vertical "connection line" visual linking parent transaction to child splits

Rights Holders Tab (Admin Only)

- Summary stats: Total Holders, Total Distributed, Average per Holder, Total Allocation %
- Card grid showing each holder with: name, role, percentage badge, total received (ETH + USD), wallet address
- Animated card entry with staggered Framer Motion transitions
- **Recent Payment Splits** section showing on-chain distributions with live indicator

Reports Tab (Admin Only)

- Report Generator for creating revenue reports

Distribute Tab (Admin Only)

- Revenue distribution panel connected to MetaMask
- Requires selecting a specific project (not "All Projects")
- Sends ETH through the smart contract and records to Supabase

3.4 Navigation Bar

Global navbar (hidden on landing, login, and signup pages):

- LUNIM logo with "Creative Platform" subtitle and green online indicator
- **Live/Demo Toggle** — Pill-style switcher between Live mode and Demo mode
- Dashboard quick-access icon
- Notification bell with red badge

- User profile dropdown with: avatar initial, name, role, links to Profile, Dashboard, Admin Panel (admin only), and Sign Out

3.5 Demo Mode

Toggled via the Live/Demo switch in the navbar:

- Persisted in localStorage
- Broadcasts state changes via CustomEvent to all components
- Filters dashboard data between real Web3 transactions and demo data
- When Demo mode is active, the toggle shows amber/orange styling
- When Live mode is active, the toggle shows violet/blue/cyan gradient

4. Smart Contract: RevenueRights.sol

A Solidity contract deployed once per project. Core logic:

- **Constructor** accepts arrays of wallet addresses, names, roles, and basis points (must sum to 10000)
- **distributeRevenue()** — Payable function that atomically splits incoming ETH proportionally
- Uses basis points for precision (e.g., 28% = 2800 basis points)
- Dust from rounding goes to the last holder
- **Events:** `HolderPaid` (per holder) and `RevenueDistributed` (per transaction)
- **receive()** fallback auto-triggers distribution on direct ETH transfers

5. Database Schema

Six tables in Supabase PostgreSQL:

Table	Key Columns	Purpose
projects	name, genre, status, contract_address, total_distributed	Creative project records
rights_holders	full_name, role, wallet_address, percentage, total_received	Creator roster per project
transactions	tx_hash, sender_address, total_amount_eth, status, method	On-chain distribution records
transaction_splits	full_name, role, percentage, amount_eth, wallet_address	Per-holder payment breakdown
users_profile	role (admin/rights_holder), display_name	User account and permissions
settings	key, value	Platform config (e.g., demo_mode)

6. Real-Time Data Sync

The dashboard maintains live data through three mechanisms:

1. **CustomEvent (payment-recorded)** — Dispatched after a successful distribution
2. **BroadcastChannel (lunim-realtime)** — Cross-tab sync within the same browser
3. **Supabase Realtime** — Postgres changes on `transactions` and `transaction_splits` tables trigger automatic dashboard refresh

7. Transaction Flow

When admin triggers a distribution:

1. Smart contract `distributeRevenue()` called via MetaMask
 2. Transaction hash captured immediately
 3. `POST /api/transactions/initiate` — Creates pending record in Supabase
 4. `await tx.wait()` — Waits for on-chain confirmation
 5. Parse `HolderPaid` events from receipt logs
 6. `POST /api/transactions/confirm` — Saves splits, updates holder/project totals
 7. Frontend refetches project data via `/api/dashboard`
 8. Realtime channels push updates to all open sessions
-

8. API Routes

The application uses Next.js API routes:

Route	Purpose
<code>/api/dashboard</code>	Main data endpoint (projects, holders, transactions)
<code>/api/projects</code>	Project CRUD operations
<code>/api/rights</code>	Rights holder management
<code>/api/revenue</code>	Revenue data with protocol bridge
<code>/api/web3/auto-distribute</code>	Automated distribution with DB sync
<code>/api/auth</code>	Authentication endpoints
<code>/api/activities</code>	Activity feed data
<code>/api/reports</code>	Report generation
<code>/api/payments</code>	Payment processing
<code>/api/stripe</code>	Stripe fiat demo layer

Plus an Express.js backend server on port 4000 with JWT middleware, rate limiting, and CORS.

9. Seeded Demo Data

7 projects with 41 total rights holders, using Hardhat test accounts:

Project	Genre	Status	Holders
Neon Requiem	Sci-Fi Thriller	Active	5 (100%)
Dust & Dynasty	Historical Drama	Active	6 (100%)
Glass Republic	Political Thriller	Active	6 (100%)
The Salt Coast	Crime Drama	Active	6 (100%)
Binary Fault	Tech Thriller	Active	6 (100%)

Velvet Underground	Crime Musical Drama	Upcoming	6 (100%)
After the Meridian	Psychological Drama	Upcoming	6 (100%)

Each project's holder percentages sum to exactly 100%.

10. Setup & Running

```
# Install dependencies
npm install

# Start Hardhat blockchain node (Terminal 1)
npm run chain

# Compile + Deploy + Seed everything (Terminal 2)
npm run demo:full

# Start full application (Terminal 3)
npm run dev
# Starts: Hardhat node + Express backend + Next.js frontend

# Run a test distribution
npm run demo 0.1
```

MetaMask Setup:

- Network: Hardhat Localhost
 - RPC URL: <http://127.0.0.1:8545>
 - Chain ID: 31337
 - 20 test accounts with 10,000 ETH each (free, no gas fees)
-

11. Development Milestones

v1.0 — Core Integration

- 0xSplits protocol integration and smart contract deployment
- Dual-module architecture (Admin vs Creator workspaces)
- Revenue management with roster and withdrawal flows
- Premium UI polish with Framer Motion animations
- Dashboard-to-blockchain real-time sync

v2.0 — Platform Maturation (April 2026)

- WalletConnect v2 multi-wallet support and ENS resolution
- CSV roster import with bulk insert capability
- Chart.js analytics dashboard and CSV/PDF report exports
- Notification engine and gas-optimized distribution batching
- Light/Dark "Soft Indigo" theme with persistent UI state

v3.0 — Production Stability (May 2026)

- Demo mode localized to navbar toggle
- Revenue distribution logic hardened
- Framer Motion animated split visualizations

- End-to-end verification and security sweep

12. Key Design Decisions

Decision	Rationale
One smart contract per project	Isolates financial flows for clean audit trails
Basis points (out of 10000)	Eliminates floating-point errors in on-chain math
Supabase with RLS	Database-level security enforcement per user role
Live/Demo toggle in navbar	Keeps production dashboard clean while enabling walkthroughs
Three-layer real-time sync	CustomEvent + BroadcastChannel + Supabase Realtime for reliability

13. Project File Structure

```
web3-distribution/
├── contracts/
│   ├── RevenueRights.sol      # Main distribution contract
│   └── RevenueSplitter.sol    # Alternative splitter
├── scripts/
│   ├── deploy.js              # Contract deployment
│   ├── seed.js                # Database seeding (7 projects, 41 holders)
│   ├── seed_users.js          # User account seeding
│   ├── demo_distribution.js    # CLI demo runner
│   └── verifyDb.js            # Database health check
├── server/
│   ├── index.js               # Express server entry
│   ├── middleware/             # JWT, rate-limit, validation
│   └── routes/                 # REST API handlers
├── src/app/
│   ├── page.tsx               # Landing page
│   ├── layout.tsx             # Root layout (AuthProvider, WalletProvider)
│   ├── login/page.tsx         # Authentication page
│   ├── dashboard/page.tsx     # Main dashboard (5 tabs)
│   ├── components/
│   │   ├── Navbar.tsx         # Global navigation bar
│   │   └── dashboard/
│   │       ├── ChartsPanel.tsx # Revenue analytics charts
│   │       ├── RevenueSnapshot.tsx # Transaction list with split expansion
│   │       ├── DistributePanel.tsx # MetaMask distribution interface
│   │       ├── RecentActivity.tsx # Activity feed
│   │       ├── ReportGenerator.tsx # PDF/CSV report builder
│   │       └── AddRightsHolderModal.tsx
│   └── lib/
│       ├── auth.tsx           # Auth context and provider
│       ├── wallet.tsx          # Web3 wallet context
│       ├── supabaseClient.ts   # Supabase browser client
│       ├── constants.ts        # ETH price, formatters
│       └── types.ts            # TypeScript interfaces
└── supabase/migrations/      # SQL schema migrations
```

```
|— package.json
|— hardhat.config.js
```

Creative Revenue & Rights Dashboard — Project Documentation v3.0
Generated May 5, 2026